Valora — Evaluation Report

Built-in run `eval_2026-10-09_1612` · embedding benchmark 2026-10-08 · 64 automated tests · reproducible with the commands in §9

1. What we evaluate and how

Valora is evaluated in layers. Each layer answers a different question:

Layer	Question it answers	Tooling	Needs an API key?
Built-in evaluation	Does the system retrieve the right comparables and produce accurate, grounded valuations?	`backend/evals/run_eval.py`, 60-case golden set + human-corrected feedback cases	No
Embedding-model benchmark	Which encoder gives the best retrieval for this data?	`backend/evals/benchmark_embeddings.py`	No
Price-model validation	How accurate is the ML predictor on unseen sales?	5-fold cross-validation in `backend/app/agents/pricing.py`	No
LLM-graded evaluation	Is the recommendation faithful and relevant, as judged by an LLM?	DeepEval (`backend/evals/run_deepeval.py`)	Yes
Online LLM-as-judge	Is *this specific* answer acceptable?	Judge agent on every request	Optional (rules fallback)
Automated tests	Do the API, guardrails, retrieval and failure paths behave correctly?	pytest, 64 tests	No
UI end-to-end	Does the product work for a user?	Scripted browser run, 15 steps, screenshots	No

Hold-out principle. Every valuation case is built from a real sale. That sale is **excluded from its own comparables** (`exclude_ids`), so the system must value the home from *other* sales, exactly as in practice. Its actual sale price is then the ground truth.

2. Golden set

`backend/evals/golden_set.json` (generated by `build_golden_set.py`) holds 60 cases:

Kind	Cases	What is checked
Valuation	44	Natural-language queries generated from real records, stratified across neighborhoods and price bands. Checked: estimate vs actual sale price, range coverage, comparable relevance, groundedness, intent.
Retrieval / intent	11	Similar-property, comparison, price-difference and market questions. Checked: intent and comparable relevance.
Guardrail	5	Prompt injection, off-topic and too-short inputs (must be blocked) plus valid queries (must pass).

Every valuation a user corrects through `POST /api/v1/feedback` is **appended automatically** as an extra case in the next run. The run below contains 61 cases: 60 golden + 1 human correction.

3. Built-in evaluation results

Rules engine (deterministic, no LLM), 61 cases, FAISS vector store, run `eval_2026-10-09_1612`. Each hold-out subject is excluded from its own comparables **and** from the pricing model's training data, so neither half of the ensemble has seen the sale it is scored on.

Metric	Definition	Score	Threshold	Cases passing
Valuation MAPE	mean |estimate − actual| / actual	**9.6 %**	≤ 15 %	80 %
Range coverage	actual price inside the recommended low–high range	**80 %**	≥ 70 %	80 %
Contextual precision	share of returned comparables in the expected neighborhood with living area within ±25 %	**91 %**	≥ 70 %	88 %
Contextual recall	at least 3 relevant comparables returned (an appraisal needs ≥ 3), scored min(1, relevant/3)	**97 %**	≥ 70 %	93 %
Faithfulness (groundedness)	share of figures in the answer that trace to a retrieved record or computed value (±1 %)	**100 %**	≥ 85 %	100 %
Hallucination rate	1 − faithfulness	**0 %**	≤ 10 %	100 %
Answer relevancy	answer names the right neighborhood and gives a price when one was asked	**100 %**	≥ 80 %	100 %
Intent accuracy	classifier intent = expected intent	**98 %**	≥ 85 %	98 %
Guardrail accuracy	block / pass decision = expected	**100 %**	≥ 95 %	100 %

Latency over the run: **p50 124 ms, p95 156 ms** per full pipeline (rules engine, Apple-silicon laptop). **All thresholds met.** The runner exits non-zero on any regression, so it can gate CI.

LLM-engine run. A stratified 18-case subset was also run with the Claude engine enabled (`eval_2026-10-08_0651`, `source: builtin+llm`; this run predates the 2026-10-09 reliability fixes and was not repeated, to avoid spending LLM credits): faithfulness 100 %, answer relevancy 100 %, contextual precision 96 %, contextual recall 100 %, MAPE 7.8 %, range coverage 83 %, intent accuracy 100 %,

guardrails 100 %. Latency rose to p50 3.4 s (three LLM calls per query). The number-producing path is identical in both engines, so the valuation figures barely move. The LLM improves phrasing and intent handling on unusual wording.

Judge agreement

The online judge's verdict is compared with an oracle: "approve if and only if the estimate is within 15 % of the true price". Agreement is **76 %** in rules mode (34 of 45 valuation cases) and **83 %** with the LLM engine (earlier run). The 11 rules-mode disagreements split evenly: 5 where the judge flagged a valuation that was in fact within 15 %, and 6 where it approved one that was not. The judge cannot see the true price. It judges evidence quality (coverage, model agreement, adjustments, grounding, range width), so a well-supported but wrong estimate can pass. This is the main motivation for the human feedback loop, and improving judge calibration is listed as future work.

4. Price-prediction model

HistGradientBoostingRegressor on log(SalePrice) with the 12 key numeric features plus neighborhood median $/sq ft and median price. Metrics come from **5-fold cross-validation (out-of-fold predictions)**, not the training fit. The two neighborhood columns are target statistics, so inside each fold they are recomputed from the training rows only; computing them over all rows leaked each held-out sale's own price into its features (measured effect: MAPE 9.52 % → 9.57 %, small but now honest):

Metric	Value
MAPE	**9.6 %**
R^2 (log scale)	**0.89**
Residual σ (log)	0.138 → used for the 90 % prediction interval
Training rows	2,927

The final valuation is an **ensemble** of the comparable-weighted estimate and this model. The comparable weight runs from 0.25 to 0.75, depending on in-neighborhood coverage and comparable dispersion. When the two methods diverge, the risk score rises.

5. Embedding-model evaluation

Four dense encoders were benchmarked in the production `HybridIndex` on the 55 golden queries that name a neighborhood (top-10 results, **no metadata filters**, so the encoder alone decides, hold-out). A result counts as relevant if it is in the expected neighborhood with living area within ±25 %.

Encoder	Dim	Dense P@10	Dense MRR	Dense hit@3	Hybrid P@10	Hybrid hit@3	Hybrid comp-APE	Query latency
LSA (TF-IDF → SVD), fallback	256	0.566	0.723	0.800	0.647	0.855	18.3 %	5 ms
all-MiniLM-L6-v2	384	0.534	0.695	0.691	0.636	0.782	21.0 %	7 ms

Encoder	Dim	Dense P@10	Dense MRR	Dense hit@3	Hybrid P@10	Hybrid hit@3	Hybrid comp-APE	Query latency
bge-small-en-v1.5 (chosen)	384	**0.649**	**0.753**	**0.836**	**0.658**	**0.873**	**18.4 %**	10 ms
bge-base-en-v1.5	768	0.627	0.742	0.782	0.634	0.818	19.2 %	27 ms
BM25 only (reference)	—	—	—	—	0.658	0.873	19.8 %	5 ms

comp-APE = error of the median sale price of the top-5 results vs the subject's true price: a direct measure of how useful the retrieved comparables are for valuation.

Findings

- **bge-small-en-v1.5 is the best encoder on every ranking metric** (precision@10, MRR, hit@3, both dense and hybrid). It beats the twice-as-large bge-base at 2.5× lower query latency and about a third of the memory. On comparable-price error the four encoders are within about one point of each other once fused with BM25 (18.3–19.2 %, MiniLM 21.0 %), so the larger model buys nothing here.
- all-MiniLM-L6-v2 is the weakest on every metric.
- **Hybrid fusion helps every encoder.** BM25 alone is strong because queries contain exact tokens (neighborhood codes, years). Fusing it with bge-small keeps BM25's precision while finding comparables whose prices are closer to the truth (comp-APE 18.4 % vs 19.8 % for BM25 alone). That is why the system is hybrid rather than keyword-only.
- These are *unfiltered* retrieval numbers. In production, hard metadata filters and the Comp Analyst's appraisal re-ranking raise contextual precision to 91 % (§3).
- The LSA fallback is respectable (hybrid P@10 0.647), so the service degrades gracefully if the neural model cannot be downloaded.

Vector store. Dense vectors are served from a FAISS `IndexFlatIP` (exact inner product on L2-normalised vectors = cosine), with metadata filters pushed into the search through an `IDSelectorBatch`. Replacing the previous numpy matrix with FAISS was verified to be **result-identical**: the built-in evaluation gives the same precision, recall and MAPE under both stores.

6. LLM-graded evaluation (DeepEval)

`backend/evals/run_deepeval.py` turns the same golden cases into DeepEval `LLMTestCase`s. The retrieved comparable records are the `retrieval_context`, and the system answer is the `actual_output`. It scores:

DeepEval metric	What it measures	Threshold
Faithfulness	claims in the answer are supported by the retrieved comparables	≥ 0.85
Answer Relevancy	the answer addresses the question asked	≥ 0.80
Contextual Precision	relevant comparables are ranked above irrelevant ones	≥ 0.70
Contextual Recall	the retrieved comparables contain what an ideal answer needs	≥ 0.70

DeepEval metric	What it measures	Threshold
Hallucination	answer contradicts the context	≤ 0.10
ValuationMAPE (custom)	estimate error vs actual sale price	≤ 0.15

Claude is the grader. The run needs an API key and costs gateway credit, so it is executed on demand (`python evals/run_deepeval.py 10` grades the first 10 cases). It writes results in the same format as the built-in runner, so the Evaluation dashboard displays either. **RAGAS** was not added separately: its core metrics (faithfulness, answer relevancy, context precision and recall) are the same family DeepEval already covers, and a second LLM-graded framework would double the grading cost without new signal.

7. Automated tests

`pytest` runs **64 tests in about 5 seconds**, offline, with the LLM disabled. All pass.

Area	Tests	Highlights
End-to-end API	10	full pipeline, metadata filters as hard constraints, search, predict, valuation, compare, properties, feedback → evaluation loop, problem-details errors
Vector store	3	FAISS results equal exact numpy search; filter (allowed-id) enforcement; empty pools; live index reports FAISS
Guardrails	8	PII redaction for phone / SSN / email; injection and off-topic blocking; control-character stripping; truncation
Attribute extraction	4	multi-digit bedrooms, half baths, "built in" vs "1850 sq ft", decades; out-of-range clamping
Pipeline edge cases	3	filters matching nothing; thin evidence → higher risk and lower confidence; price-difference intent
API boundaries	10	`top_k` bounds, missing / overlong query, invalid filters, negative areas, invalid feedback, same-record compare, feedback for an unknown request → 404 with nothing stored
LLM wrapper	6	JSON extraction from fenced / prose replies, chat-format structured output, one repair call on invalid JSON, circuit breaker on billing errors, fallback model → `llm-degraded`, rules mode without a key
LangGraph orchestration	12	topology; judge-driven retry runs once with a wider pool; guardrail early exit; optional Risk Analyst failure tolerated; a critical failure (Retriever, Pricing Model, Valuer, Judge) stops the run with no estimate; a guardrail crash fails closed; an over-generous LLM judge cannot overturn a deterministic FAIL; grounding catches `$512k` shorthand
Data integrity & intents	8	zero bedrooms / basement / baths kept (priors only for missing values); "no basement" / "no garage"; first-mentioned neighborhood is the subject; intent-matched answers stay grounded; price-basis note; CV features use training rows only

Bugs found by these tests and fixed:

1. "12 bedrooms" was parsed as **2** (the pattern took one digit). Bedroom, bath and garage patterns now accept two digits.
2. "2.5 baths" was parsed as **5**. Half baths are now handled.

3. An SSN was redacted but **labelled as a phone number** (the patterns ran in the wrong order). The SSN now runs before the looser phone pattern.
4. `POST /api/v1/predict` and `/valuation` **accepted negative areas and counts**. Explicit-attribute requests now reject them with `422`. Values extracted from text are still clamped, not rejected.

8. UI end-to-end test

A scripted browser session (`docs/testing/report.html`, screenshots in `docs/testing/`) exercised 15 user journeys against the live backend. **All 15 passed:** homepage, workspace navigation, a valuation query ($204,000, 8 comparables), comparable drill-down (semantic / BM25 / hybrid scores, $ adjustments), feedback with a corrected value, metadata filters (Old Town, built ≤ 1940, ≤ 3 beds → all comparables comply), prompt-injection blocked, compare with hedonic attribution, property index search and sort, evaluation dashboard, API reference, dark mode, mobile layout at 390 px, API error formats, and the health endpoint.

9. Reproducing these results

```
cd backend && source .venv/bin/activate
pytest -q                                  # 64 tests
python evals/run_eval.py                   # built-in evaluation → evals/results/latest.json
python evals/benchmark_embeddings.py       # embedding benchmark →
evals/results/embedding_benchmark.json
pip install -r requirements-eval.txt
ANTHROPIC_API_KEY=... python evals/run_deepeval.py 10      # LLM-graded, first 10 cases (costs
credit)
```

10. Limitations

- The golden set is generated from the same dataset the system searches. Hold-out exclusion prevents self-matching, but it is not a test on a different city or time period.
- Sales span 2006–2010. Comparables are not time-adjusted (only a recency bonus), so valuations are in 2006–2010 dollars.
- Answer relevancy in the built-in runner is a lexical proxy. DeepEval's LLM-graded relevancy is the stronger measure.
- Thin neighborhoods (Green Hills 2 sales, Landmark 1) give low confidence by design. The risk score and judge surface this rather than hide it.
- Comparable adjustments use fixed coefficients (quality 5.5 %/point, age 0.25 %/year, garage 4 %/bay, basement, living area at 0.55 × the comp's $/sq ft). Replacing them with the hedonic model's fitted elasticities was tested on 400 leave-one-out sales: mean comps-only error improved (11.6 % → 11.2 %) but median error worsened (6.8 % → 6.9 %), so the simpler fixed values were kept.
- Every intent runs through the same valuation pipeline. Similar-property, market and comparison questions get an answer framed for that intent, but there is no time-series trend analysis, and two-property comparisons are handled by the separate `/api/v1/compare` endpoint (Compare page).